Redis Caching for RecruitmentDb

Task Submission | 27 October 2026

1. Objective

Implement and verify Redis caching for an applicant application-count endpoint. SQL Server remains the authoritative data source. Repeated reads use Redis, committed application writes invalidate cached counts, and Redis failures fall back to SQL Server.

2. Implemented system

Component	Implementation
API	Node.js 22+, built-in HTTP server, mssql and redis clients
SQL Server	Existing RecruitmentDb in rootless Podman
Redis	Official Redis 8 Alpine container; 60-second cache expiry
Test page	Local browser interface with cache source, timings, and SQL query totals
Access	API: 127.0.0.1:3101; Redis: 127.0.0.1:6379

3. Request flow

```
Browser / HTTP client
          |
          v
Applicant count API -----> Redis GET
          |                   |
          |                   +-- HIT: return cached JSON
          |
          +-- MISS / unavailable: query SQL Server
                       |
                       v
          fn_ApplicantApplicationCount(applicant_id)
                       |
                       +-- store in Redis with expiry
                       +-- return applicant and count
```

4. API routes

Method	Route / purpose
GET	/api/applicants/:id/application-count - cached count
POST	/api/applicants/:id/applications - atomic application + skill
POST	/api/applicants/:id/cache - clear this applicant cache
GET	/api/applicants/:id/options - unused jobs and skills
GET	/api/metrics and /health - counters and service readiness

5. Cache-aside implementation

Each count request first checks Redis. A hit returns the stored JSON and skips the SQL count query. A miss calls the existing scalar function, then stores the response using SET with EX = 60 seconds. The response contains applicant_id, full_name, and application_count.

```
SELECT applicant_id, full_name,
       dbo.fn_ApplicantApplicationCount(applicant_id)
           AS application_count
FROM dbo.APPLICANT
WHERE applicant_id = @id;
```

The @id value is bound as a SQL integer parameter. Redis keys include a unique API-process namespace, applicant ID, and application-count suffix. Redis is temporary: no persistent volume or disk persistence is used.

Response header	Meaning
X-Cache: MISS	SQL Server supplied the count; Redis stored the response.
X-Cache: HIT	Redis supplied the response; no SQL count query ran.
X-Cache: BYPASS	Cache was unavailable or invalid; SQL supplied the count.

6. Invalidation after application creation

POST calls dbo.usp_ApplyForJobAndAddSkill. The stored procedure inserts an application and an existing catalog skill association in one transaction. The application trigger also creates the initial Submitted event. Redis invalidation occurs only after the procedure commits.

A duplicate application, duplicate applicant skill, or missing referenced ID causes rollback. Failed writes retain valid cached counts. Successful writes delete the affected applicant key, so the next count is refreshed from SQL Server.

7. Failure handling and concurrency

If Redis is unavailable, the API returns a fresh SQL result with BYPASS. A committed write still returns HTTP 201 if cache deletion fails. The key is marked dirty and must be deleted before it can be reused after Redis recovers.

A per-applicant lock serializes count reads and application writes inside this API process. This prevents an earlier count query from filling the cache with an old value after a write. A new namespace on API restart avoids reusing stale entries from a previous instance.

8. Local request protection

Mutation routes require a random per-process bearer token supplied automatically by the local page. Cross-origin browser requests are rejected. Database credentials are read on the server and are not included in browser responses or the submission. The API is a local demonstration, not a public authentication service.

9. Automated verification

The completed integration suite passed all 11 tests, with zero failures. Tests used a separate RecruitmentRedisTest_* database and a separate temporary Redis container. Cleanup removed both. Original RecruitmentDb row counts and checksums were unchanged by that test run.

Test	Verification	Result
1	MISS then HIT; one SQL count query	PASS
2	TTL expiry causes another SQL refresh	PASS
3	Committed write invalidates cache and creates initial status	PASS
4	Duplicate skill rolls back the application without eviction	PASS
5	Duplicate application and foreign-key error responses	PASS
6	Zero counts, missing applicants, and invalid input	PASS
7	Mutation token and cross-origin protections	PASS
8	Concurrent reads and writes avoid stale fills	PASS
9	Corrupt cache values fall back to SQL	PASS
10	Real Redis stop/restart; outage writes and fresh recovery	PASS
11	Original recruitment database remains unchanged	PASS

10. Live demonstration evidence

Request	HTTP status	X-Cache	Application count
1	200	MISS	11
2	200	HIT	11

Observed for applicant 1 on 27 October 2026, after clearing its key. Both reads returned the same count. This is a live sample-data snapshot, not a fixed expected value for future requests. No performance improvement percentage is claimed.

11. Run and reproduce

From the supplied recruitment-api folder, with the existing sqlserver-test container and RecruitmentDb workflow objects available:

```
npm ci
./start.sh
# Open http://127.0.0.1:3101/
# Click Clear this cache, then Read twice.
npm test
```

The browser should show MISS then HIT. Test TTL is shortened to 2 seconds; normal API TTL remains 60 seconds. See README.md and TEST-RESULTS.txt in the source folder for setup details and the complete test output.

12. Scope and limitations

Direct SQL/Adminer writes require manual clearing or cache expiry. Locks and dirty-key tracking cover one API process; multiple instances need shared invalidation coordination. Manual application creation commits to the sample database; automated test writes use a separate disposable database.